

Chapter 4: Threads

Operating System Concepts

Based on Silberschatz, Galvin and Gagne – 9th Edition
Complete Study Notes — Easy to Understand, First Time

1. Introduction to Threads

DEFINITION

A thread is a lightweight process. It is the smallest unit of execution within a process.

What Is a Thread? (Simple Explanation)

Imagine you are running a music player app on your phone. The app is doing three things at the same time:

- Playing the song (audio)
- Showing the album art and updating the progress bar (UI)
- Downloading the next song in the background (network)

Each of these tasks is a thread. They all belong to the same app (process), but they run independently.

Without threads, the app would have to do one thing at a time — play the song, THEN update the screen, THEN download the next song. That would be terrible!

KEY IDEA

A process is the app. Threads are the individual workers inside the app. Every thread runs one step at a time (sequentially) on its own, but many threads running together makes the whole program feel fast and responsive.

How Does a Thread Work?

- Each thread runs strictly one instruction after another — it is sequential.

- Every thread has its own Program Counter (PC) — this tells the thread which instruction to run next.
- Every thread has its own Stack — this stores local variables and tracks which function the thread is currently inside.
- Threads share the CPU, just like processes do. One thread runs, then another — they take turns.
- A thread can create child threads (just like a process can create child processes).
- A thread can block — meaning it pauses to wait for something (e.g., reading a file from disk).

Single Thread vs Multiple Threads

A traditional (heavyweight) process has only ONE thread — it does one thing at a time.

If you give a process MULTIPLE threads, it can do many things at the same time — much more powerful!

Concept	Explanation	Important Note
Thread	Lightweight process	Smallest unit of execution
Program Counter	Tracks current instruction	Each thread has its OWN
Stack	Stores local variables	Each thread has its OWN
Shared CPU	Threads take turns using CPU	Like process scheduling
Child Thread	A thread created by another thread	Similar to child processes
Heavyweight Process	Traditional process with one thread	Can only do one task at a time

2. Thread Characteristics

DEFINITION

Thread characteristics are the properties that define how threads behave and what they share or keep private.

Shared Memory — All Threads Live in the Same House

All threads inside the same process share the exact same address space. Think of it like this:

Imagine 4 people (threads) living in the same house (process). The house has a living room, kitchen, and storage room. Everyone can use them. But each person has their own personal room (stack) and their own diary (program counter).

- Threads share: Code section (the program instructions)
- Threads share: Data section (global variables)
- Threads share: OS resources like open files and signals
- Each thread owns: Its own Program Counter and Stack

⚠ IMPORTANT — DANGER OF SHARING

Because all threads share the same memory space, one thread CAN read, write, or even completely wipe out another thread's stack!

This is why multithreaded programs must be very carefully designed. This is also why we need synchronization techniques (covered in later chapters).

Thread States

Just like a process, a thread can be in different states at different times:

Thread State	What It Means	Real-Life Analogy
Running	Thread is currently using the CPU and executing instructions	A worker actively typing on a keyboard
Blocked	Thread is waiting for something (e.g., I/O, system call)	A worker waiting for a printer to finish
Ready	Thread wants to run but is waiting for its turn on the CPU	A worker in a queue, ready to work
Terminated	Thread has finished all its work and is done	A worker who has clocked out and gone home

What Threads Share vs What They Own

Shared (Between ALL Threads)	Private (Each Thread Owns)
Code section (program instructions)	Program Counter (PC)
Data section (global variables)	Stack (local variables and function calls)
Open files	Register set
Signals	Thread ID

3. Thread vs Process — The House Analogy

💡 WHY THIS ANALOGY?

The house analogy is one of the best ways to understand threads and processes. Once you understand this, the whole chapter becomes much clearer.

The Analogy Explained

Process = A House

A house is a container. It has attributes — bedrooms, a kitchen, a living room, floor space. But the house itself doesn't DO anything on its own. It just sits there. This is exactly what a process is — a passive container of resources (memory, files, code).

Threads = The People Living in the House

The people (occupants) are the ACTIVE ones. They cook, watch TV, take showers, sleep. They are the ones actually doing work. Threads are like the people — they are the active workers inside the process.

Single Threaded = Living Alone

If you live alone in the house, life is simple. You do anything you want, any time you want. You use the kitchen when you like, the bathroom when you like — no coordination needed. But you can only do ONE thing at a time. This is a single-threaded process.

Multi Threaded = Living With Others

When you get married and your spouse moves in, things change! Now two people share the same house (same resources). You can cook while your spouse watches TV — two tasks happening at the same time! But now you must coordinate — you can't both use the bathroom simultaneously. This is a multi-threaded process.

Analogy	OS Concept	Behavior
The House	Process	Passive container with attributes and resources
Occupants (People)	Threads	Active workers doing actual tasks
Lone occupant (alone)	Single Thread	One task at a time, no sharing needed
Multiple occupants (family)	Multiple Threads	Concurrent tasks, shared resources, need coordination

4. Single Thread vs Multi Thread

DEFINITION

Single Thread: A process with only one thread — one task at a time.

Multi Thread: A process with multiple threads — multiple tasks simultaneously.

Visual Diagrams

Single-Threaded Process

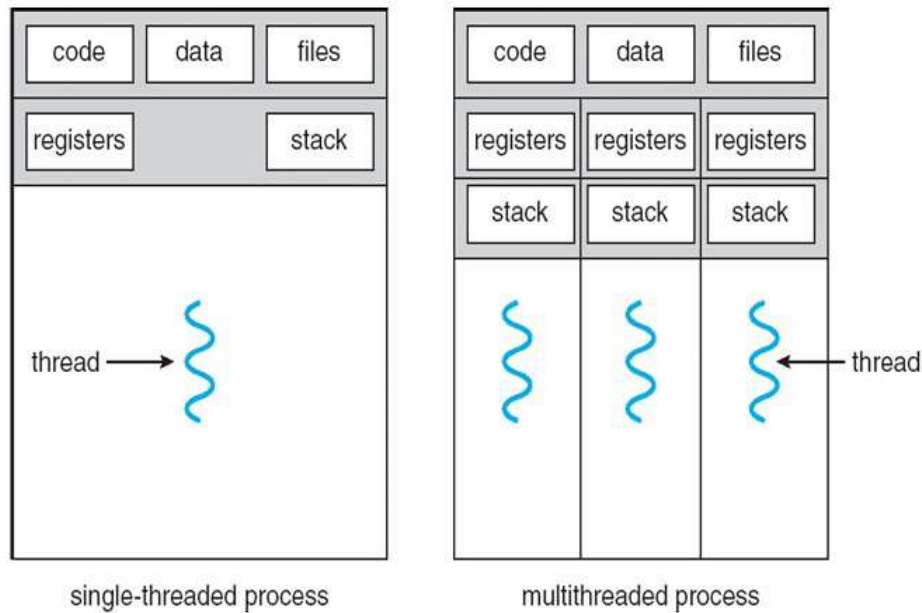
In a single-threaded process, there is only ONE thread. That thread has its own registers and stack, and it shares code, data, and files with the process.

SINGLE-THREADED PROCESS		
code	data	files (shared by process)
registers	stack	(belonging to the 1 thread)
↑ Only ONE thread runs here		

Multi-Threaded Process

In a multi-threaded process, multiple threads share the same code, data, and files — but each thread has its OWN registers and stack. They all run inside the same process.

MULTI-THREADED PROCESS		
code	data	files (shared by ALL threads)
registers	registers	registers (each thread has its own)
stack	stack	stack (each thread has its own)
Thread 1 Thread 2 Thread 3 (all running inside one process)		



Single Thread	Multi Thread
Only 1 thread per process	Multiple threads per process
One task at a time	Many tasks at the same time
If blocked → whole process waits	Other threads continue even if one blocks
Simple, easy to manage	Complex but very powerful and efficient
Traditional / heavyweight process	Modern, lightweight approach

5. Motivation — Why Use Threads?

The Web Server Problem

Let's say you are building a web server — like the server that runs Google or YouTube. Thousands of users send requests every second.

? PROBLEM — Without Threads

If the web server uses a single thread, it can only handle ONE user at a time.

User 1 sends a request → server processes it (takes 2 seconds) → then User 2's request is processed → then User 3... and so on.

If 1000 users send requests at the same time, User 1000 has to wait 2000 seconds (over 33 minutes!) just to get a web page. Completely unusable.

OLD SOLUTION — Multiple Processes

One idea was to create a new PROCESS for each client request.

Problem: Creating a process is SLOW (has to set up memory, copy resources, etc.) and uses a LOT of resources. Not practical for thousands of requests per second.

BEST SOLUTION — Multiple Threads

Instead of creating a new process, create a new THREAD for each request.

Creating a thread is much faster and uses far fewer resources than creating a process.

The server has one main thread listening for requests. When a request arrives, it spawns a new lightweight thread to handle it. The main thread immediately goes back to listening. All requests are handled simultaneously!

Web Server Flow — Step by Step

Step	What Happens
Step 1	Client request arrives at the web server
Step 2	The main listening thread receives the request
Step 3	The server spawns (creates) a NEW THREAD to handle that request
Step 4	The new thread processes the request and sends back the response
Step 5	Meanwhile, the main thread is already handling the NEXT request
Result	Thousands of requests handled simultaneously — fast and efficient!

6. Benefits of Using Threads

There are four main benefits of using threads instead of processes. Remember these — they are frequently asked in exams!

BENEFIT 1: Responsiveness

The program keeps responding to the user even when part of it is busy or blocked.

Real-life example: You are using a web browser and click to load a large image. Without threads, the browser would freeze and you could not do anything until the image loads. With threads, one thread loads the image while another thread keeps the browser UI responsive — you can still click other things, type in the address bar, etc.

BENEFIT 2: Resource Sharing

Threads automatically share the memory and resources of their parent process.

Real-life example: If you have 5 threads in a word processing app, they all share the same document data in memory. No need to copy the document for each thread. Compare this to creating 5 separate processes — you would need to send copies of data between them (which is slow and complicated).

BENEFIT 3: Economy

Creating and switching between threads is MUCH cheaper than creating and switching between processes.

Why? A new process needs its own memory space, file descriptors, security info, etc. — all copied from the parent. A new thread just needs a stack and registers — it reuses everything else from the process.

Real-life comparison: Creating a process is like building a brand new house. Creating a thread is like adding a new person to an existing house. Much easier and cheaper!

BENEFIT 4: Utilization of Multiprocessor Architectures

Modern computers have multiple CPU cores. Threads can run in PARALLEL — one thread on each CPU core simultaneously.

Real-life example: Rendering a 3D movie involves billions of calculations. With multiple threads on multiple cores, different parts of the movie can be calculated simultaneously, reducing render time from weeks to hours.

Benefit	Key Idea	Real-World Example
Responsiveness	Program keeps running even when part is blocked	Browser stays clickable while image loads
Resource Sharing	Threads share memory automatically	5 threads work on same document in memory
Economy	Cheaper to create & switch than processes	Thread = new person; Process = new house
Multiprocessor Use	Threads run in parallel on multiple CPUs	3D rendering split across many cores

7. Thread Library

DEFINITION

A thread library is a set of code (an API) that programmers use to create and manage threads in their programs.
Think of it as a toolbox for threads — it provides the tools (functions) to create a thread, destroy a thread, make a thread wait, etc.

Two Types of Thread Libraries

There are two main ways to implement a thread library, and this leads to the two types of threads: User Threads and Kernel Threads.

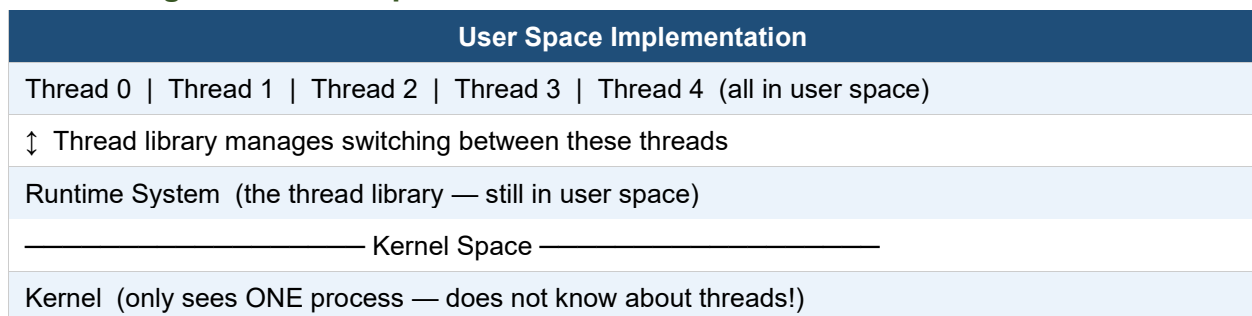
Feature	User Thread (Type 1)	Kernel Thread (Type 2)
Where managed?	In user space (by a library, not the OS)	In kernel space (by the OS directly)
OS awareness	OS does NOT know threads exist	OS fully knows and manages threads
System call needed?	No — local function call in user space	Yes — every operation needs a system call
Speed	Very fast to create and switch	Slower — kernel overhead involved
Code/data location	All in user space	All in kernel space

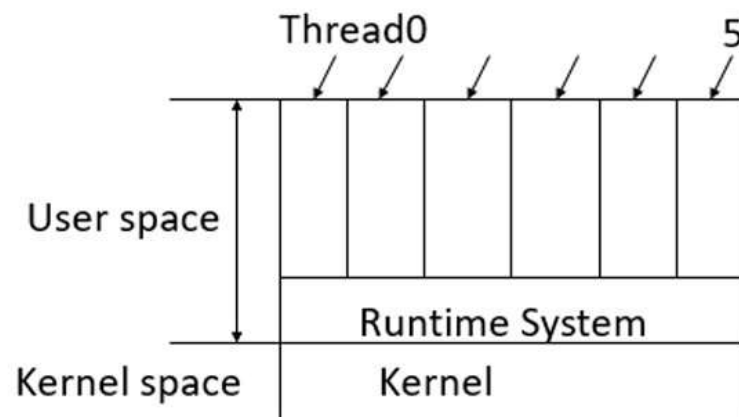
8. User Threads — Implementation in User Space

How User Threads Work

With user threads, the operating system has NO idea that threads exist. From the OS's perspective, there is only ONE process running. The thread management is done entirely by a thread library running in user space.

Visual Diagram — User Space Threads





Advantages of User Threads

Advantage	Explanation	Why It Matters
Fast creation	Creating/destroying threads is cheap — all done in user space, no kernel needed	No overhead from system calls
Works on any OS	Can be used even on OS that does not support threads (e.g., old UNIX)	Portable across different systems
Very fast context switch	Only CPU registers need to be saved/loaded — no need to change MMU or flush TLB	Much faster than switching processes
Scales well	No kernel tables needed — can have huge numbers of threads	Kernel threads use limited kernel memory

Disadvantages of User Threads

Disadvantage	Explanation
Blocking system calls stop ALL threads	If Thread A makes a blocking call (e.g., read file), the kernel blocks the ENTIRE process — all threads stop, even ones that could run!
Page faults block everything	If a thread causes a page fault, the kernel blocks the whole process — it doesn't know other threads could run
No CPU preemption	If one thread starts running, it runs until it voluntarily gives up the CPU — no thread can interrupt it
Bad for CPU-bound tasks	Threads are useful when some block while others run. If threads never block, there's no benefit

⚠ THE BIG PROBLEM VISUALIZED

Thread A makes a blocking system call (reads from disk)...

- Kernel blocks the ENTIRE process
- Thread B, C, D are all frozen — even though THEY have work to do!
- Kernel does not know Thread B, C, D even exist (they are hidden in user space)

This is the fundamental weakness of user threads.

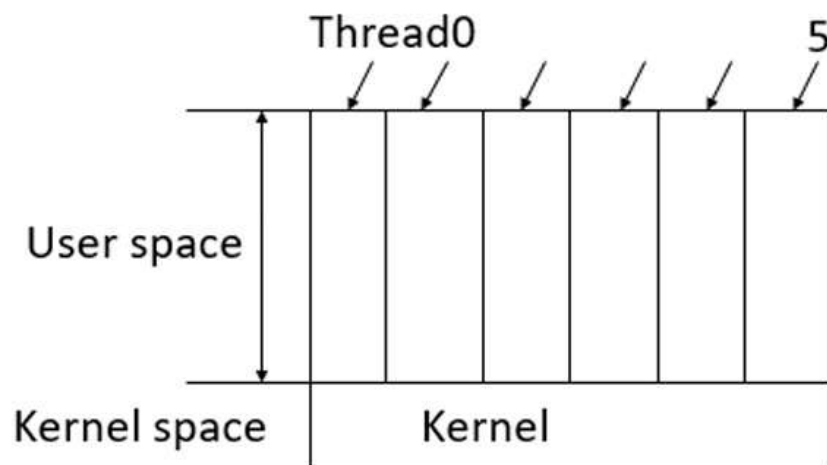
9. Kernel Threads — Implementation in Kernel Space

How Kernel Threads Work

With kernel threads, the operating system KNOWS about every thread. The kernel itself manages thread creation, scheduling, and deletion. No separate thread library in user space is needed.

Visual Diagram — Kernel Space Threads

Kernel Space Implementation					
Thread 0	Thread 1	Thread 2	Thread 3	Thread 4	(visible to kernel)
↑ Threads exist in both user space and kernel space					
User Space (thread stacks, code)					
———— Kernel Space —————					
Kernel (knows ALL threads, manages scheduling, keeps thread table)					



How the Kernel Manages Threads

- The kernel keeps ONE table per process.
- Each table has ONE ENTRY per thread.
- When a thread wants to create or destroy another thread → it makes a KERNEL CALL.
- The kernel does the actual creation/destruction.
- When one thread blocks → the kernel can immediately run another thread from the same process or even a different process.

Advantages of Kernel Threads

Advantage	Explanation
Kernel manages everything	No runtime system needed in user space — kernel handles all thread operations directly
Blocking handled properly	When one thread blocks, kernel runs another thread — no need to freeze the whole process
True parallelism possible	Threads from the same process can run on DIFFERENT CPU cores simultaneously
Better for blocking tasks	I/O-heavy programs work much better — blocked threads don't hold others up

Disadvantages of Kernel Threads

Disadvantage	Explanation
Every operation needs system call	Create, delete, synchronize threads — all require a system call to the kernel. System calls are slow!
Context switch is expensive	Switching between kernel threads is as costly as switching between entire processes
Speed advantage disappears	The main reason to use threads over processes (speed) is largely lost due to kernel overhead

10. User Thread vs Kernel Thread — Full Comparison

Feature	User Thread	Kernel Thread
Who manages?	User-space library	OS Kernel directly
System call needed?	NO — local function call	YES — every operation
Speed of creation	FAST (cheap)	SLOW (expensive)
Context switch speed	Very fast (only registers)	Slow (like process switch)

Blocking problem?	YES — blocks ALL threads	NO — kernel schedules others
True parallelism?	NO (only 1 runs at a time)	YES (can use multiple CPUs)
OS awareness?	OS does NOT know threads	OS fully manages threads
Scalability	HIGH (no kernel tables needed)	LIMITED (uses kernel memory)
Works without OS support?	YES	NO

11. Quick Revision — One Page Summary

Concept	Key Point
Thread =	Lightweight process. Smallest execution unit inside a process.
Thread has its OWN:	Program Counter, Stack, Registers
Thread SHARES:	Code, Data, Files, Global Variables
Thread States:	Running → Ready → Blocked → Terminated
Why threads?	Cheap, Fast, Efficient for concurrent tasks
Motivation:	Web servers need to serve thousands of clients simultaneously
Benefit 1: Responsiveness	Program keeps running even when part is blocked
Benefit 2: Resource Sharing	Memory shared automatically — no copying needed
Benefit 3: Economy	Thread creation is cheaper than process creation
Benefit 4: Multiprocessor	Threads run in parallel on multiple CPUs
Thread Library:	API for creating & managing threads (user or kernel level)
User Thread:	Managed in user space — fast but blocks all on system call
Kernel Thread:	Managed by OS — slower but handles blocking properly
User Thread +:	Fast creation, fast switch, works without OS support
User Thread -:	Blocking system call stops ALL threads, no true parallelism
Kernel Thread +:	Kernel-aware, blocking handled, true parallelism possible
Kernel Thread -:	Every operation = system call = slow, switch as expensive as process

12. Top 5 Most Important Questions & Answers

HOW TO USE THIS SECTION

These are the 5 most commonly asked questions from this chapter. Read the question first, think about your answer, then read the detailed answer below. If you can answer all 5 confidently, you are fully prepared for this chapter.

Q1. What is a thread? How is it different from a process?

Answer:

A thread is the smallest unit of execution within a process. It is called a 'lightweight process' because it shares most of the resources of its parent process, making it much cheaper to create and manage than a full process.

Process	Thread
A heavy, independent unit of execution	A lightweight unit inside a process
Has its OWN complete memory space	Shares memory with other threads in same process
Creating a process is expensive	Creating a thread is fast and cheap
Processes don't share memory by default	All threads in a process share code, data, and files
Context switching is slow	Context switching between threads is fast
One process can have multiple threads	A thread always belongs to exactly one process

Key technical detail: Each thread has its own Program Counter (to track where it is in the code) and its own Stack (to store local variables). Everything else is shared.

Q2. What are the four benefits of using threads? Explain each with an example.

Answer:

1. Responsiveness: A program continues running even if part of it is blocked. Example: A web browser can still respond to clicks while loading a large image in the background (image loading is one thread, UI response is another).

2. Resource Sharing: Threads share the process's memory automatically. Example: Multiple threads in a word processor all work on the same document data in memory — no need to send copies of data between them.

3. Economy: Threads are cheaper to create and switch between than processes, because they reuse the process's resources. Example: A web server creates a new thread (not a new process) for each client request — much faster.

4. Utilization of Multiprocessor Architectures: In a computer with multiple CPU cores, different threads can run on different cores at the same time. Example: A 3D rendering program can split the rendering work across 8 cores, running 8 threads in parallel and finishing 8x faster.

Q3. What is a Thread Library? Explain User Threads and Kernel Threads.

Answer:

A thread library is an API (Application Programming Interface) that gives programmers functions to create, manage, and destroy threads. There are two types:

User Thread: The thread library runs entirely in user space. The OS does not know threads exist — it only sees one process. Thread management (scheduling, switching) is done by the library itself. This makes user threads very fast to create and switch. However, if one user thread makes a blocking system call, the kernel blocks the entire process — all other threads freeze.

Kernel Thread: The OS kernel directly manages the threads. Every thread operation (create, delete, synchronize) requires a system call to the kernel. This is slower, but the kernel can properly handle blocking — when one thread blocks, the kernel runs another thread from the same process. True parallel execution on multiple CPUs is also possible.

Q4. What are the advantages and disadvantages of User Threads and Kernel Threads?

Answer:

User Threads — Advantages:

- Cheap and fast to create and destroy (no kernel needed)
- Very fast context switching — only CPU registers need saving
- Works on any OS, even systems that do not support threads at the kernel level
- Can scale to very large numbers of threads without using kernel memory

User Threads — Disadvantages:

- If one thread makes a blocking system call → the ENTIRE process (all threads) blocks
- Page faults also block all threads — kernel does not know about individual threads
- No true parallelism — only one thread runs at a time

Kernel Threads — Advantages:

- Kernel knows all threads → when one blocks, another can run immediately
- True parallelism: different threads can run on different CPU cores at the same time
- Better handling of blocking operations

Kernel Threads — Disadvantages:

- Every thread operation (create, delete, sync) requires a slow system call
- Context switching is as expensive as switching between processes
- Much of the performance benefit of threads over processes disappears

Q5. Explain the motivation for using threads in a web server. Why not use multiple processes?

Answer:

A web server must handle many client requests simultaneously. Consider a server with thousands of users all requesting pages at the same time.

Problem with a Single-Threaded Server: If the server has only one thread, it can serve only ONE client at a time. Client 2 must wait for Client 1 to be fully served before being attended to. With thousands of clients, wait times become enormous — completely impractical.

Why Not Multiple Processes? One solution is to create a new process for each request. But process creation is time-consuming and resource-intensive — it requires allocating new memory space, copying resources, setting up file descriptors, etc. Doing this thousands of times per second is too slow and too expensive.

Best Solution — Multiple Threads: The server runs as one process with multiple threads. One main thread listens for incoming requests. When a request arrives, the server creates a new lightweight thread to handle it. That thread serves the client, then terminates. The main thread immediately returns to listening. Because thread creation is very cheap, this scales to thousands of simultaneous clients efficiently.

In summary: Threads give the server the concurrency it needs, without the high cost of process creation. This is why virtually all modern web servers (Apache, Nginx, etc.) use a multi-threaded or event-driven model.